GPCell: Gaussian-process oscillation detection for single-cell time series

Tristan Sones-Dykes

2026-04-11

Summary

GPCell is a Python package for classifying single-cell gene expression time series as oscillatory or non-oscillatory using Gaussian process models. It supports the canonical workflow introduced by Phillips et al. (2017): load live-imaging traces, estimate background noise, detrend each cell, fit a non-oscillatory Ornstein-Uhlenbeck (OU) model and an oscillatory OU-times-cosine model, and compare the fitted models to identify candidate oscillatory cells.

The package provides two user-facing interfaces. `OscillatorDetector` preserves a compact class-based workflow for existing scripts and notebooks, including the `from_file(...)` entry point used for the bundled Hes1 example data. The newer functional API exposes typed dataclasses such as `DetectorConfig`, `DetectorData`, `BICResult`, and `PipelineResult`, with `run_pipeline(...)` as the main orchestration function for new code. The stable production paths are the BIC workflow and the parametric-bootstrap workflow for thresholding log-likelihood ratios. A fully Bayesian MCMC path remains available in the class interface for experimental work, but it is not the primary supported route in this JOSS submission.

Statement of need

Single-cell live imaging can produce short, noisy, irregular biological time series in which visual inspection and classical periodicity tests are unreliable. Phillips et al. (2017) addressed this by comparing GP models derived from an OU process and a quasi-periodic OU-cosine covariance, providing an objective way to ask whether each trace is better explained as oscillatory or aperiodic. The original implementation was MATLAB-centered, which limits accessibility for Python-based computational biology workflows and makes it harder to reuse the method within modern testing, packaging, and continuous-integration practices.

GPCell fills this software gap by reworking the Phillips-style oscillation detector as an installable Python package built around GPflow (Matthews et al. 2017), NumPy (Harris et al. 2020), SciPy (Virtanen et al. 2020), TensorFlow (Abadi et al. 2016), and standard Python development tooling. Its target users are computational biologists analyzing dynamic reporter traces, researchers reproducing or extending GP-based oscillation detection, and developers who need a reusable GPflow implementation rather than a single-purpose notebook.

State of the field

General-purpose Gaussian process libraries such as GPflow (Matthews et al. 2017), GPyTorch (Gardner et al. 2018), and GPstuff (Vanhatalo et al. 2012) provide flexible GP modelling primitives, but they do not provide an end-to-end single-cell oscillation classification workflow with domain-specific preprocessing, replicate fitting, OU versus OU-cosine model comparison, and bootstrap-based thresholding. Conversely, the Phillips MATLAB code is method-specific but tied to an older environment and is less naturally integrated with Python data tooling and automated tests.

GPCell therefore follows a build-on-top strategy rather than replacing existing GP frameworks. It delegates GP regression and optimization to GPflow while contributing the domain workflow: CSV conventions for cell and background traces, background-noise estimation, RBF detrending, replicate fitting, BIC and log-likelihood-ratio summaries, synthetic non-oscillatory bootstrap traces, and regression tests against included reference data. This makes the package narrower than a general GP library and more reusable than a dissertation-era analysis script.

Software design

The central design decision in GPCell is to separate domain orchestration from GP construction. At the backend layer, `GPRConstructor` turns a kernel specification, prior factory, trainable parameter map, kernel-composition operator, and inference mode into a concrete GPflow model. This allows the same fitting utilities to instantiate a single OU kernel or a composite OU-times-cosine kernel while preserving explicit control over priors and trainable parameters. The `GaussianProcess` wrapper then standardizes fitting, prediction, log-posterior extraction, and optional sampling helpers around GPflow models.

At the workflow layer, the functional API makes the stable production path easier to test: `prepare_detector_data(...)` estimates noise and detrends traces, `fit_bic(...)` fits OU and OU-cosine replicate models and returns typed BIC results, and `fit_bootstrap(...)` generates synthetic non-oscillatory traces for thresholding. `run_pipeline(...)` composes these steps without exposing mutable detector state. The class API delegates the BIC and

bootstrap paths to this functional layer while preserving the historical field names and plotting hooks used by existing notebooks.

GPCell also addresses the practical cost of fitting many exact GP models. Each cell is fitted independently across multiple model replicates, making it a natural target for process-based parallelism. The `fit_processes_joblib(...)` path uses Joblib (Varoquaux et al. 2024) to distribute per-trace work and uses memory-mapped arrays for homogeneous or padded trace collections to reduce worker-process data copying. Simulation utilities use Numba (Lam, Pitrou, and Seibert 2015) for performance-sensitive stochastic data generation, which supports reproducible bootstrap and benchmark workflows without changing the exact maximum-likelihood GP inference model used in the stable classifier.

Research impact statement

GPCell packages a previously dissertation-centered implementation into reusable research software: it has project metadata in `pyproject.toml`, an MIT license, CI-backed linting and tests, generated API documentation, regression tests tied to the included Hes1 example, and hand-written architecture and development notes. The included tests exercise both compatibility behavior and the newer functional API, helping preserve the scientific behavior of the Phillips-style workflow while changing the implementation language and architecture.

The near-term scholarly value is in reproducibility and extensibility. Researchers can run the Hes1 example workflow from a clean Python environment, inspect the GPflow model construction logic, and extend priors or fitting utilities without rewriting the full detector. GPCell does not claim new biological discoveries in this JOSS paper; it contributes maintainable software for performing and extending an established GP-based oscillation-classification method.

AI usage disclosure

This draft paper and associated JOSS-preparation metadata were assisted by OpenAI Codex in April 2026. The assistance was limited to repository inspection, drafting, editing, and formatting support. The submitting author is responsible for reviewing, editing, validating, and approving the final paper and for the scientific and software-design decisions described here.

Acknowledgements

This work originated as an MMath dissertation project at the School of Mathematics and Statistics, University of St Andrews. The author acknowledges the foundational methodological contribution of Phillips et al. (2017). No external funding is declared in this draft.

References

Abadi, Martín, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, et al. 2016. "TensorFlow: A System for Large-Scale Machine Learning." In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 265–83. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi.

Gardner, Jacob R., Geoff Pleiss, David Bindel, Kilian Q. Weinberger, and Andrew Gordon Wilson. 2018. "GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration." *Advances in Neural Information Processing Systems* 31. https://proceedings.neurips.cc/paper/2018/hash/27e8e17134dd7083b050476733207ea1-Abstract.html.

Harris, Charles R., K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, et al. 2020. "Array Programming with NumPy." *Nature* 585: 357–62. https://doi.org/10.1038/s41586-020-2649-2.

Lam, Siu Kwan, Antoine Pitrou, and Stanley Seibert. 2015. "Numba: A LLVM-Based Python JIT Compiler." In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, 1–6. https://doi.org/10.1145/2833157.2833162.

Matthews, Alexander G. de G., Mark van der Wilk, Tom Nickson, Keisuke Fujii, Alexis Boukouvalas, Pablo León-Villagrá, Zoubin Ghahramani, and James Hensman. 2017. "GPflow: A Gaussian Process Library Using TensorFlow." *Journal of Machine Learning Research* 18 (40): 1–6. https://doi.org/10.48550/arXiv.1610.08733.

Phillips, Nick E., Cerys Manning, Nancy Papalopulu, and Magnus Rattray. 2017. "Identifying Stochastic Oscillations in Single-Cell Live Imaging Time Series Using Gaussian Processes." *PLOS Computational Biology* 13 (5): e1005479. https://doi.org/10.1371/journal.pcbi.1005479.

Vanhatalo, Jarno, Jaakko Riihimäki, Jouni Hartikainen, Pasi Jylänki, Ville Tolvanen, and Aki Vehtari. 2012. "Bayesian Modeling with Gaussian Processes Using the GPstuff Toolbox." *arXiv*. https://doi.org/10.48550/arXiv.1206.5754.

Varoquaux, Gaël, Olivier Grisel, Oscar Vahtras, et al. 2024. "Joblib." https://joblib.readthedocs.io/.

Virtanen, Pauli, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, et al. 2020. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." *Nature Methods* 17: 261–72. https://doi.org/10.1038/s41592-019-0686-2.